Tutorial

Giolit SystemVerilog Assertion Verifier: proving SVA properties of RTL designs, step by step

Contents

1. What the verifier does 2

2. Before you start 3

3. A first proof 3

4. Reading a counterexample 4

5. Memories and invariants 5

6. When a property fails, check the property 6

7. Timing windows 6

8. State machines 7

9. Assumptions: what the environment promises 7

10. Liveness: something good eventually happens 8

11. Hierarchy, parameters, and what to do with UNDECIDED 9

12. Real designs 10

13. What a result means 10

14. Results in scripts and CI 11

15. Certificates and evidence 11

16. Reading a proof 12

- 16.1 What the verifier proves . . . 12
- 16.2 The names in a proof . . . 13
- 16.3 The shape of a proof . . . 13
- 16.4 A small proof, checked by hand . . . 13
- 16.5 A proof that lists every reachable state . . . 14

16.6 A proof by an invariant the prover found 14
16.7 Proof kinds 14

17. Safety margins and the Giolit Safety Resilience Score **15**

18. The SDK: the verifier in your own tools **16**

19. Command reference **17**

1. What the verifier does

The **Giolit SystemVerilog Assertion Verifier** (`sva-verify`) takes an RTL design written in SystemVerilog (IEEE 1800-2017), together with the assertions written in it, and decides each assertion **for every possible input sequence and every possible start state**. It does not simulate test vectors: it reasons about all runs of the design at once.

Input. One or more `.sv`/`.v` files: synthesizable RTL with SystemVerilog Assertions (SVA). Every `assert property`, `cover property` and immediate `assert` in the design is checked. Every `assume property` constrains the inputs.

Output. One line per property, with its status, and a trace for every failure or cover:

Status	Meaning
PROVED	No run of the design violates the assertion. This is a proof, not a test result.
FAILED	Some run violates it. The run is printed clock cycle by clock cycle.
COVERED	A cover property can happen; the witness run is printed.
UNREACHABLE	A cover property can never happen (dead logic, or over-constrained inputs).
REACHED	A liveness goal is reached on some run, but not proved for every run.
UNDECIDED	Neither proved nor refuted within the time budget; the reason is printed.
UNSUPPORTED	The property uses a construct the verifier does not model; the reason is printed.

The command exits with **0** when every assertion is proved, **1** when one fails, **2** when some are undecided or unsupported, **3** for a usage or input error and **4** without a valid license, so it fits into scripts and CI.

How it decides. Three engines work on the same model of the design at the same time, and the first to settle a property decides it:

- a **prover** that computes invariants of the design (facts true in every reachable state) and refines them until they exclude the failure;
- an **explorer** that enumerates the reachable states of designs whose state space is small enough, and finds the loops that refute liveness properties;
- a **simulator** that runs the design on random inputs and catches deep bugs quickly.

Every FAILED or COVERED result comes with a real run of the design. Every PROVED result is backed by an invariant or by an exhaustive exploration.

2. Before you start

Install the verifier as described in the README and check it:

```
sva-verify --version
sva-verify --license      # runs and days left in the trial (uses no run)
```

The tutorial examples are installed in `share/giolit/examples/` under the installation folder (by default `~/.local/giolit-sva-verify/share/giolit/examples`). Copy them to a working folder and run the commands of this tutorial there:

```
cp -r ~/.local/giolit-sva-verify/share/giolit/examples ~/sva-tutorial
cd ~/sva-tutorial
```

File	Shows
`01_counter.sv`	a first proof; a cover
`02_counter_bug.sv`	reading a counterexample
`03_fifo.sv`	memories, pointer invariants
`04_arbiter.sv`	a property that is wrong, not the design
`05_handshake.sv`	timing windows (`##[1:3]`)
`06_traffic_light.sv`	an enum state machine
`07_hierarchy.sv`	hierarchy, `generate`, parameters; data-width reduction
`08_assume.sv`	assumptions on the inputs
`09_liveness.sv`	liveness: `s_eventually`, lassos
`10_lockstep.sv`	a proof by an inductive invariant (section 16)
`11_margins.sv`	safety margins and the resilience score (section 17)

3. A first proof

`01_counter.sv` is a counter that counts from 0 to 9 and wraps:

```
module counter (
    input  logic       clk,
    input  logic       rst,
    input  logic       en,
    output logic [3:0] cnt
);
  always_ff @(posedge clk)
    if (rst)     cnt <= 4'd0;
    else if (en) cnt <= (cnt == 4'd9) ? 4'd0 : cnt + 4'd1;

  default clocking cb @(posedge clk); endclocking
  default disable iff (rst);

  p_range: assert property (cnt <= 4'd9);
  p_wrap:  assert property (en && cnt == 4'd9 |=> cnt == 4'd0);
  p_hold:  assert property (!en |=> $stable(cnt));
  c_wrap:  cover  property (cnt == 4'd9 ##1 cnt == 4'd0);
endmodule
```

Verify it:

```
$ sva-verify --reset=rst 01_counter.sv
Giolit SystemVerilog Assertion Verifier 1.0.0 — top (auto), 4 properties

  PROVED      assert    counter.p_range  (01_counter.sv:16)
```

```
  PROVED      assert    counter.p_wrap  (01_counter.sv:17)
  PROVED      assert    counter.p_hold  (01_counter.sv:18)
  COVERED     cover     counter.c_wrap  (01_counter.sv:19)
      witness, 12 cycles (cycle 0 is the first clock edge):
      cycle 0  : cnt=15 en=1 rst=1
      cycle 1  : cnt=0 rst=0
      cycle 2  : cnt=1
      cycle 3  : cnt=2
      cycle 4  : cnt=3
      cycle 5  : cnt=4
      cycle 6  : cnt=5
      cycle 7  : cnt=6
      cycle 8  : cnt=7
      cycle 9  : cnt=8
      cycle 10 : cnt=9
      cycle 11 : cnt=0

4 properties: 3 proved, 0 failed, 1 covered, 0 unreachable covers, 0 liveness reached (not proved), 0
    undecided, 0 unsupported  (0.1 s)
Result: ALL ASSERTIONS PROVED
```

All three assertions are proved: no input sequence can drive `cnt` above 9, make it skip the wrap, or change it while `en` is low. The cover property `c_wrap` is **COVERED**, and its witness shows a run in which the counter wraps.

Why `--reset=rst`? Registers start with arbitrary values: the verifier does not assume that a simulator's initial zeros exist in silicon. `--reset=rst` says that `rst` is active in the first clock cycle, as after power-on; after that it is a free input like any other (an active-low reset is written `--reset='!rst_n'`). Without it, the first cycle may start with `cnt = 15`, and `p_range` fails right away:

```
$ sva-verify 01_counter.sv
Giolit SystemVerilog Assertion Verifier 1.0.0 — top (auto), 4 properties

  FAILED      assert    counter.p_range  (01_counter.sv:16)
      counterexample, 1 cycle (cycle 0 is the first clock edge):
      cycle 0  : cnt=15 en=1 rst=0
  PROVED      assert    counter.p_wrap  (01_counter.sv:17)
  PROVED      assert    counter.p_hold  (01_counter.sv:18)
  COVERED     cover     counter.c_wrap  (01_counter.sv:19)
      witness, 2 cycles (cycle 0 is the first clock edge):
      cycle 0  : cnt=9 en=1 rst=0
      cycle 1  : cnt=0

4 properties: 2 proved, 1 failed, 1 covered, 0 unreachable covers, 0 liveness reached (not proved), 0
    undecided, 0 unsupported  (0.1 s)
Result: ASSERTION FAILURES FOUND
```

This is the most common surprise for new users. If a property fails in cycle 0 or 1 with values a reset would have cleared, add `--reset`.

`disable iff (rst)` switches the assertions off while the reset is active. `default disable iff` applies it to every assertion of the module.

4. Reading a counterexample

`02_counter_bug.sv` has a seeded bug: it wraps at 10 instead of 9.

```
$ sva-verify --reset=rst 02_counter_bug.sv
Giolit SystemVerilog Assertion Verifier 1.0.0 — top (auto), 1 property

  FAILED      assert    counter_bug.p_range  (02_counter_bug.sv:13)
      counterexample, 12 cycles (cycle 0 is the first clock edge):
      cycle 0  : cnt=15 en=1 rst=1
      cycle 1  : cnt=0 rst=0
      cycle 2  : cnt=1
      cycle 3  : cnt=2
      cycle 4  : cnt=3
      cycle 5  : cnt=4
```

```
      cycle 6  : cnt=5
      cycle 7  : cnt=6
      cycle 8  : cnt=7
      cycle 9  : cnt=8
      cycle 10 : cnt=9
      cycle 11 : cnt=10

1 properties: 0 proved, 1 failed, 0 covered, 0 unreachable covers, 0 liveness reached (not proved), 0
    undecided, 0 unsupported  (0.0 s)
Result: ASSERTION FAILURES FOUND
```

Read the trace from the top:

- **cycle 0 is the first clock edge.** Each line shows the design's signals at that cycle, after the clock edge.
- **Only the signals that change are printed**, after the first line. Here `en` stays 1 the whole time, so it appears only in cycle 0.
- The **last cycle is the one in which the assertion fails**: `cnt = 10` violates `cnt <= 9`.

The trace is a real run of the design, usually a short one. Use `--verbose` to also print each property's source text and full traces of more than 40 cycles. To replay a trace in your simulator, drive the inputs (`rst`, `en`) with the values of each cycle.

`--json=FILE` writes the same results, traces included, as JSON (section 14).

5. Memories and invariants

`03_fifo.sv` is a synchronous FIFO of depth 8 with an occupancy counter. Its assertions state invariants: the counter stays in range, `full` and `empty` are exclusive, and the pointers agree with the counter. The cover property asks whether the FIFO can become full.

```
$ sva-verify --reset='!rst_n' 03_fifo.sv
Giolit SystemVerilog Assertion Verifier 1.0.0 — top (auto), 6 properties

  PROVED      assert    fifo.a_bound  (03_fifo.sv:40)
  PROVED      assert    fifo.a_not_both  (03_fifo.sv:41)
  PROVED      assert    fifo.a_ptr_sync  (03_fifo.sv:42)
  PROVED      assert    fifo.a_no_ovf  (03_fifo.sv:43)
  PROVED      assert    fifo.a_no_unf  (03_fifo.sv:44)
  COVERED     cover     fifo.c_full  (03_fifo.sv:45)
      witness, 15 cycles (cycle 0 is the first clock edge):
      cycle 0  : count=11 pop=1 push=0 rp=1 rst_n=0 wp=4
      cycle 1  : count=0 rp=0 wp=0
      cycle 2  : (no change)
      cycle 3  : (no change)
      cycle 4  : (no change)
      ... 6 more cycles
      cycle 11 : count=5 wp=5
      cycle 12 : count=6 wp=6
      cycle 13 : count=7 wp=7
      cycle 14 : count=8 wp=0

6 properties: 5 proved, 0 failed, 1 covered, 0 unreachable covers, 0 liveness reached (not proved), 0
    undecided, 0 unsupported  (0.4 s)
Note: asynchronous resets are modelled as synchronous to the clock (async2sync)
Result: ALL ASSERTIONS PROVED
```

The proof holds for every content of the memory and every sequence of pushes and pops. Note `a_ptr_sync`:

```
a_ptr_sync: assert property (AW'(wp - rp) == AW'(count));
```

Invariants like this one, relating several registers, are the core of most proofs. If one of your properties is UNDECIDED, adding the invariant your design relies on as an extra assertion often helps you see what is going on, and it documents the design.

6. When a property fails, check the property

A FAILED result means the property is false for the design as written. Often the design is wrong; sometimes the property says something you did not mean. `04_arbiter.sv` is a round-robin arbiter for two clients, with two attempts at stating fairness:

```
// "when both request, client 0 is granted within two cycles"
a_fair_naive: assert property (req[0] && req[1] |-> ##[1:2] gnt[0]);
// "a client that requests in two consecutive cycles is granted in the second one or the next"
a_fair:       assert property (req[0] ##1 req[0] |-> ##[0:1] gnt[0]);
```

```
$ sva-verify --reset=rst 04_arbiter.sv
Giolit SystemVerilog Assertion Verifier 1.0.0 — top (auto), 4 properties

  PROVED      assert    arbiter.a_onehot  (04_arbiter.sv:31)
  PROVED      assert    arbiter.a_only_req  (04_arbiter.sv:32)
  FAILED      assert    arbiter.a_fair_naive  (04_arbiter.sv:35)
      counterexample, 5 cycles (cycle 0 is the first clock edge):
      cycle 0  : gnt=3 last=1 req=3 rst=1
      cycle 1  : gnt=0 rst=0
      cycle 2  : gnt=1 last=0
      cycle 3  : gnt=2 last=1 req=2
      cycle 4  : req=3
  PROVED      assert    arbiter.a_fair  (04_arbiter.sv:37)

4 properties: 3 proved, 1 failed, 0 covered, 0 unreachable covers, 0 liveness reached (not proved), 0
    undecided, 0 unsupported  (0.1 s)
Result: ASSERTION FAILURES FOUND
```

The trace of `a_fair_naive` shows the reason: in the cycle after both clients request, client 1 is granted, and client 0 then withdraws its request, so it is never granted. The arbiter is fair; the naive property asked for something no arbiter can promise to a client that gives up. `a_fair` states what was meant, and it is proved.

When a trace looks impossible, check three things in this order: the property (does it say what you mean?), the reset and the start state (section 3), and the inputs (does the trace use input values your environment never produces? then add an assumption, section 9).

7. Timing windows

SVA describes time with sequences. `05_handshake.sv` is a responder that acknowledges a request after a few cycles:

```
a_ack_in_time:        assert property (!busy && req |-> ##[1:3] ack);
a_ack_only_after_req: assert property (ack |-> !$past(ack));
a_bug:                assert property (!busy && req |-> ##1 ack);
```

```
$ sva-verify --reset=rst 05_handshake.sv
Giolit SystemVerilog Assertion Verifier 1.0.0 — top (auto), 3 properties

  PROVED      assert    handshake.a_ack_in_time  (05_handshake.sv:29)
  PROVED      assert    handshake.a_ack_only_after_req  (05_handshake.sv:30)
  FAILED      assert    handshake.a_bug  (05_handshake.sv:31)
      counterexample, 3 cycles (cycle 0 is the first clock edge):
      cycle 0  : ack=1 busy=1 req=1 rst=1 wait_cnt=3
      cycle 1  : ack=0 busy=0 rst=0 wait_cnt=0
      cycle 2  : busy=1

3 properties: 2 proved, 1 failed, 0 covered, 0 unreachable covers, 0 liveness reached (not proved), 0
    undecided, 0 unsupported  (0.0 s)
Result: ASSERTION FAILURES FOUND
```

`##[1:3] ack` means "`ack` holds in one of the next 1 to 3 cycles": the responder needs two cycles, so the window holds and the one-cycle version `a_bug` fails, with the trace showing the request and the missing `ack`.

Operator	Meaning
`a ##1 b`	`b` one cycle after `a`
`a ##[m:n] b`	`b` between `m` and `n` cycles after `a` (`$` for no bound)
`a [*n]`, `a [*m:n]`	`a` repeated for consecutive cycles
`a [->n]`, `a [=n]`	the `n`-th occurrence of `a` (goto / non-consecutive)
`S |-> P`	whenever sequence `S` matches, property `P` starts in its last cycle
`S |=> P`	`P` starts in the cycle after the match
`$past(e, n)`, `$rose`, `$fell`, `$stable`, `$changed`	values in earlier cycles

8. State machines

`06_traffic_light.sv` is a controller for two crossing roads, written as an `enum` state machine. It shows the usual checks of a controller: a safety property (never both green), a timing bound, and a cover property that confirms an important transition is possible.

```
$ sva-verify --reset=rst 06_traffic_light.sv
Giolit SystemVerilog Assertion Verifier 1.0.0 — top (auto), 4 properties

  PROVED      assert    traffic_light.a_never_both_green  (06_traffic_light.sv:31)
  PROVED      assert    traffic_light.a_yellow_short  (06_traffic_light.sv:32)
  COVERED     cover     traffic_light.c_ns_green  (06_traffic_light.sv:33)
      witness, 15 cycles (cycle 0 is the first clock edge):
      cycle 0  : car_ew=1 rst=1 state=3 timer=7
      cycle 1  : rst=0 state=0 timer=0
      cycle 2  : timer=1
      cycle 3  : timer=2
      cycle 4  : timer=3
      ... 6 more cycles
      cycle 11 : timer=4
      cycle 12 : state=3 timer=0
      cycle 13 : timer=1
      cycle 14 : state=0 timer=0
  FAILED      assert    traffic_light.a_seeded_bug  (06_traffic_light.sv:34)
      counterexample, 12 cycles (cycle 0 is the first clock edge):
      cycle 0  : car_ew=1 rst=1 state=3 timer=7
      cycle 1  : rst=0 state=0 timer=0
      cycle 2  : timer=1
      cycle 3  : timer=2
      cycle 4  : timer=3
      cycle 5  : state=1 timer=0
      cycle 6  : timer=1
      cycle 7  : state=2 timer=0
      cycle 8  : timer=1
      cycle 9  : timer=2
      cycle 10 : timer=3
      cycle 11 : timer=4

4 properties: 2 proved, 1 failed, 1 covered, 0 unreachable covers, 0 liveness reached (not proved), 0
    undecided, 0 unsupported  (0.1 s)
Result: ASSERTION FAILURES FOUND
```

`a_seeded_bug` claims that the east–west green phase lasts at most three cycles; the trace shows the timer reaching four. Covers matter as much as assertions: an assertion is trivially true on a design that can never reach the interesting states, and a COVERED result shows that it can.

9. Assumptions: what the environment promises

A design is usually correct only for the inputs its environment produces. **`assume property`** tells the verifier which inputs those are. `08_assume.sv` is a one-slot buffer whose producer must wait for `ready`:

```
m_wait_ready:   assume property (put |-> ready);
a_nothing_lost: assert property (!lost);
a_hold:         assert property (full && !take && !put |=> full && $stable(data_out));
c_refill:       cover  property (take ##1 put);
```

```
$ sva-verify --reset=rst -G W=2 08_assume.sv
Giolit SystemVerilog Assertion Verifier 1.0.0 — top (auto), 3 properties

  PROVED      assert    one_slot.a_nothing_lost  (08_assume.sv:33)
  PROVED      assert    one_slot.a_hold  (08_assume.sv:34)
  COVERED     cover     one_slot.c_refill  (08_assume.sv:35)
      witness, 3 cycles (cycle 0 is the first clock edge):
      cycle 0  : data_in=3 data_out=3 full=1 lost=1 put=1 rst=1 take=1
      cycle 1  : full=0 lost=0 put=0 rst=0
      cycle 2  : put=1

3 properties: 2 proved, 0 failed, 1 covered, 0 unreachable covers, 0 liveness reached (not proved), 0
    undecided, 0 unsupported  (0.1 s)
Result: ALL ASSERTIONS PROVED
```

Without the assumption, a producer that ignores `ready` overwrites data, and the verifier shows how (the file leaves the assumption out when `NO_ASSUME` is defined):

```
$ sva-verify --reset=rst -G W=2 -D NO_ASSUME 08_assume.sv
Giolit SystemVerilog Assertion Verifier 1.0.0 — top (auto), 3 properties

  FAILED      assert    one_slot.a_nothing_lost  (08_assume.sv:33)
      counterexample, 4 cycles (cycle 0 is the first clock edge):
      cycle 0  : data_in=3 data_out=3 full=1 lost=1 put=1 rst=1 take=1
      cycle 1  : full=0 lost=0 rst=0
      cycle 2  : full=1
      cycle 3  : lost=1
  PROVED      assert    one_slot.a_hold  (08_assume.sv:34)
  COVERED     cover     one_slot.c_refill  (08_assume.sv:35)
      witness, 3 cycles (cycle 0 is the first clock edge):
      cycle 0  : data_in=3 data_out=3 full=1 lost=1 put=1 rst=1 take=1
      cycle 1  : full=0 lost=0 rst=0
      cycle 2  : full=1

3 properties: 1 proved, 1 failed, 1 covered, 0 unreachable covers, 0 liveness reached (not proved), 0
    undecided, 0 unsupported  (0.1 s)
Result: ASSERTION FAILURES FOUND
```

Assumptions are powerful and dangerous: an assumption that is too strong excludes real behaviour, and everything is then “proved” about runs that never happen. Two habits protect you:

- **Cover what matters.** `c_refill` shows that a refill is still possible under the assumption. If a cover you expect to be COVERED is UNREACHABLE, your assumptions (or your reset) exclude it.
- **Assume about inputs only**, and state each assumption as a requirement of the neighbouring block, so it can be checked there as an assertion.

`-G W=2` sets the data width to 2 bits. Section 11 explains why.

10. Liveness: something good eventually happens

Safety properties say that nothing bad ever happens. Liveness properties say that something good eventually happens, without a deadline:

```
m_hold:        assume property (req[0] && !gnt_rr[0] && !gnt_prio[0] |=> req[0]);
l_rr_served:   assert property (req[0] |-> s_eventually gnt_rr[0]);
l_prio_served: assert property (req[0] |-> s_eventually gnt_prio[0]);
```

`09_liveness.sv` serves client 0 with two arbiters: a round-robin one and a fixed-priority one in which client 1 always wins.

```
$ sva-verify --reset=rst 09_liveness.sv
Giolit SystemVerilog Assertion Verifier 1.0.0 — top (auto), 2 properties

  PROVED      liveness  liveness.l_rr_served  (09_liveness.sv:32)
  FAILED      liveness  liveness.l_prio_served  (09_liveness.sv:33)
      reason: lasso: counterexample (exhaustive state exploration) [hybrid model]
      counterexample, 5 cycles (cycle 0 is the first clock edge):
      cycle 0  : gnt_prio=1 gnt_rr=1 last=1 req=3 rst=1
      cycle 1  : gnt_prio=0 gnt_rr=0 rst=0
      cycle 2  : gnt_rr=1 last=0
      cycle 3  : gnt_rr=0 last=1 req=2
      cycle 4  : (no change)
      loop: cycles 4..4 repeat forever, the obligation never discharged

2 properties: 1 proved, 1 failed, 0 covered, 0 unreachable covers, 0 liveness reached (not proved), 0
    undecided, 0 unsupported  (21.5 s)
Result: ASSERTION FAILURES FOUND
```

- **PROVED** for a liveness property means that every pending obligation is discharged within a bounded number of cycles on every run (or that the antecedent can never match).
- **FAILED** comes with a **lasso**: a run that reaches a loop and stays in it forever without discharging the obligation. The output says which cycles repeat: here client 1 requests forever, so the fixed-priority arbiter never serves client 0.
- **REACHED** means only that some run reaches the goal. It is a witness, not a proof; it is reported when neither the proof nor a lasso is found in time.

Liveness properties usually need **fairness assumptions** like `m_hold` (a client keeps asking until served). `assume property (s_eventually e)` states that `e` happens infinitely often on the runs that matter.

11. Hierarchy, parameters, and what to do with UNDECIDED

`07_hierarchy.sv` builds a 4-stage pipeline with `generate` from a `stage` submodule, and asserts its latency:

```
a_latency: assert property (@(posedge clk) ##4 dout == $past(din, 4));
```

The verifier elaborates the whole hierarchy: parameters, `generate` loops, interfaces, packages and arrays of instances. At the full data width of 8 bits this property is beyond the time budget:

```
$ sva-verify --timeout=20 07_hierarchy.sv
Giolit SystemVerilog Assertion Verifier 1.0.0 — top (auto), 1 property

  UNDECIDED   assert    pipeline.a_latency  (07_hierarchy.sv:20)
      reason: WALL_CLOCK

1 properties: 0 proved, 0 failed, 0 covered, 0 unreachable covers, 0 liveness reached (not proved), 1
    undecided, 0 unsupported  (17.0 s)
Result: INCOMPLETE (undecided or unsupported properties)
```

A pipeline treats its data as opaque: it never inspects the bits. Its correctness therefore does not depend on the data width, and verifying it at a width of 2 bits is just as conclusive. This **data-width reduction** is a standard formal-verification technique. `-G NAME=VALUE` overrides a parameter of the top module:

```
$ sva-verify -G W=2 07_hierarchy.sv
Giolit SystemVerilog Assertion Verifier 1.0.0 — top (auto), 1 property

  PROVED      assert    pipeline.a_latency  (07_hierarchy.sv:20)

1 properties: 1 proved, 0 failed, 0 covered, 0 unreachable covers, 0 liveness reached (not proved), 0
    undecided, 0 unsupported  (0.1 s)
Result: ALL ASSERTIONS PROVED
```

When a property is UNDECIDED, try these in order:

1. **Check the reset** (`--reset`). Without it every register starts arbitrary, and invariants have to cover states the design never reaches.
2. **Give it more time**: `--timeout=300`. The budget is shared by all properties; the reason printed tells you whether time ran out.
3. **Verify the property alone**: `--property NAME` builds a model of just the logic the property depends on (its cone of influence).
4. **Reduce data widths and sizes** with `-G`: data paths, memory depths and counter widths often don't affect control properties.
5. **Constrain the inputs** with assumptions that your environment really guarantees (section 9).
6. **Add the invariants the design relies on** as assertions (section 5); each is checked, and a failing one points at the misunderstanding.
7. **Try all encodings at once**: `--encoding=race` verifies a word-level, a bit-level and a mixed model in parallel.

Properties that compare wide data registers with their past values, such as `$stable(data_out)` on a 64-bit bus, are the hardest case at full width; steps 3 and 4 are the remedy.

12. Real designs

Several files, include paths and macros:

```
sva-verify --top=soc_top -I rtl/include -D SYNTHESIS rtl/*.sv props/*.sv
```

Pass the files and options you would pass to your simulator. `--top` is needed when more than one module is uninstantiated.

Properties in a separate file. Keep the design untouched and bind a checker module into it:

```
module fifo_props (input logic clk, rst_n, full, empty);
  default clocking cb @(posedge clk); endclocking
  a_not_both: assert property (disable iff (!rst_n) !(full && empty));
endmodule
bind fifo fifo_props u_props (.*);
```

Quick checks without editing the design: `-p 'EXPR'` asserts `always EXPR` on the top module, for example `sva-verify --reset='!rst_n' -p '!(full && empty)' 03_fifo.sv`. Such an assertion has no `disable iff`: it must also hold in the reset cycle, when registers may still hold arbitrary values.

One property: `--property fifo.a_ptr_sync` (or just `a_ptr_sync`).

Several clocks. By default every clocked process runs on one global clock, as in SymbiYosys and EBMC. For designs whose behaviour depends on the relation between clocks (clock dividers, gated or derived clocks, clock-domain crossings), use `--multiclock`: then any interleaving of the clocks' edges is considered.

13. What a result means

A result holds for the **model** the verifier builds of your design:

Aspect	Model
Time	one step per clock cycle; with `--multiclock`, one step per clock edge
Inputs	free in every cycle, except as constrained by `assume property`
Start state	registers without an initial value start arbitrary; `--reset` activates a reset in the first cycle
X and Z	any 0/1 value; X in an assertion makes it fail if some choice fails

Aspect	Model
Asynchronous resets	act at the clock edge (synchronous model)
`$past` before the first cycle	0
Initial blocks	executed once before the first cycle; `initial assert` checks the start
Display and dump tasks	ignored; `$readmemh`/`$readmemb` leave the memory arbitrary (a note says so)

Supported SystemVerilog: modules, interfaces and modports, packages, parameters and `generate`, functions (input arguments), `always_ff`, `always_comb`, `always_latch`, continuous assignments, gate primitives, packed structs and unions, enums, arrays of up to 62-bit elements, `$anyconst`, `$anyseq` and `$initstate` for formal-only inputs, checkers and `bind`.

Supported SVA: concurrent and immediate assertions, assumptions and covers; sequences (`##`, `##[m:n]`, `[*]`, `[->]`, `[=]`, and, or, `intersect`, `within`, `throughout`, `first_match`); properties (`|->`, `|=>`, `#-#`, `#=#`, `not`, and, or, `implies`, `iff`, `if`/`else`, `case`, `always`, `s_always`, `nexttime`, `s_eventually`, `until`, `s_until`, `strong`, `weak`, `accept_on`, `reject_on`, `disable iff`); `default clocking` and `default disable iff`.

Not supported (reported with the reason): signals wider than 62 bits, `real` and `string` values, local variables in sequences, properties with several clocks, `$past` with a gating expression, `fork`/`join`, `wait` and event controls inside procedural code. An unsupported construct **in a property** makes only that property UNSUPPORTED; one **in the design** stops the run with exit code 3 and names the construct.

14. Results in scripts and CI

`--json=results.json` writes:

```
{
  "tool": "Giolit SystemVerilog Assertion Verifier", "version": "1.0.0",
  "design": ["02_counter_bug.sv"], "top": "", "seconds": 0.02,
  "properties": [
    { "name": "counter_bug.p_range", "kind": "assert", "status": "FAILED",
      "file": "02_counter_bug.sv", "line": 13, "seconds": 0.006,
      "source": "p_range: assert property (...)",
      "method": "counterexample (exhaustive state exploration) [hybrid model]",
      "cause": "",
      "trace": [ {"cnt": 15, "en": 1, "rst": 1}, {"cnt": 0, "en": 1, "rst": 0} ] }
  ]
}
```

`kind` is `assert`, `immediate`, `cover`, `liveness` or `strong`; `status` is one of the statuses of section 1; `trace` has one object per clock cycle with every signal's value. A CI gate needs only the exit code:

```
sva-verify --reset=rst --timeout=600 --json=sva.json rtl/*.sv
case $? in
  0) echo "all assertions proved" ;;
  1) echo "assertion failures: see sva.json"; exit 1 ;;
  2) echo "undecided properties: review sva.json" ;;
  *) echo "verification did not run"; exit 1 ;;
esac
```

15. Certificates and evidence

`--report=DIR` turns a run into evidence you can hand to a customer, an auditor or a design review. The verifier writes a bundle to `DIR`:

```
sva-verify --reset=rst --report=counter-cert --issued-to="Acme Silicon" 01_counter.sv
```

```
...
Result: ALL ASSERTIONS PROVED
Report: counter-cert (VERIFIED, document GIO-SVA-61ED05F99BE7): counter-cert/certificate.pdf
```

File	Contents
`certificate.pdf`	when every assertion is proved: the **certificate of formal verification**, with the proofs
`counterexample-report.pdf`	when an assertion fails: every failing run as a timing diagram and cycle by cycle
`verification-report.pdf`	otherwise: what is proved, what is open and why
`proof.json`	every proof in design terms and as the engine stated it, and every run
`evidence/<property>.vcd`	every counterexample and cover witness as a waveform (GTKWave or any viewer)
`evidence/<property>_replay.sv`	the same run as a testbench for your simulator
`certificate.json` (+ `.sig`)	the statement of the result, with the SHA-256 of every design file and every evidence file
`results.json`, `reproduce.sh`, the design files, `README.txt`	

The proof of a PROVED property is an *inductive invariant* over the design's state; section 16 shows how to read one. Section 17 explains the **safety margins** and the **Giolit Safety Resilience Score** on the certificate.

A counterexample replays in your simulator. The testbench forces the start state the verifier chose, drives the inputs cycle by cycle, and prints when the design's own assertion is expected to fail:

```
verilator --binary --assert --timing --top-module giolit_replay \
    02_counter_bug.sv evidence/counter_bug_p_range_replay.sv
./obj_dir/Vgiolit_replay
# [115] %Error: 02_counter_bug.sv:13: Assertion failed in giolit_replay.dut.p_range
```

Checking a bundle. With a full-edition license, `certificate.json` is signed with your organisation's key, which Giolit certified; `verify_certificate.py` in the bundle checks the signatures, the design files and every evidence file. The trial version writes the same documents, marked as trial output and unsigned. PDFs need LuaLaTeX (README, section 1).

16. Reading a proof

A proof is the strongest kind of evidence: it covers every run, for every input sequence. This section shows how to read one, so that you (or your customer's reviewer) can check it by hand.

16.1 What the verifier proves

Every proof on a certificate is a relation **Inv** over the *state of the design at a clock edge*, with three properties:

	Inv …	You can check it by asking …
1. Start	holds at the first clock edge	does every start state satisfy Inv?
2. Step	if it holds at a clock edge, it holds at the next one, for every input	from a state in Inv, can one clock cycle leave Inv?

	Inv …	You can check it by asking …
3. Safety	excludes every violation	can a cycle that starts in Inv violate the property?

Together they mean that Inv holds at *every* clock edge of *every* run, and so the property holds in every cycle. This is the same argument as a proof by induction over the clock cycles.

16.2 The names in a proof

The state is more than the design's registers:

Name	Meaning
`cnt`, `u_core.state`, `mem[3]`	a register, a register in a sub-instance, a memory word
`count[2]`	bit 2 of `count` (when the proof needs single bits)
`$past(cnt)`	the value `cnt` had at the previous clock edge (the assertions sample it)
`$initstate`	1 in the first clock cycle, 0 afterwards
`p_wrap:s1`	a state of the *checker* of property `p_wrap`: 1 while an attempt of the property is at that stage of its sequence (here: the antecedent matched one cycle ago, the consequent is due now)
`a_latency:age7`	a counter of a checker: clock cycles since an attempt started
`l_rr:pending`	1 while an obligation of the liveness property `l_rr` is pending

The PDF lists the names it uses under each proof.

16.3 The shape of a proof

A proof is printed as **relations that hold in every case**, followed by **cases**:

```
Inv = the relations that hold in every case, and one of the cases:
  in every case   r1 ∧ r2 ∧ …
  case 1          r3 ∧ r4
  case 2          r5 ∧ r6
```

Inv is `r1 ∧ r2 ∧ ... ∧ (case 1 ∨ case 2 ∨ ...)`. A relation is a bound or an equation over the names above, such as `0 <= level <= 9`, `cnt = 9`, or `wp - rp = count`.

16.4 A small proof, checked by hand

`08_assume.sv` with `-G W=2` proves `a_nothing_lost: assert property (!lost)`. Its proof has two cases:

```
case 1   $initstate = 1
case 2   $initstate = 0 ∧ lost = 0
```

1. **Start.** At the first clock edge `$initstate = 1`: case 1. ✓
2. **Step.** From case 1 (the reset cycle, `rst` = 1) the next state has `$initstate = 0` and `lost = 0` (the reset clears `lost`): case 2. From case 2, `lost` only becomes 1 on a `put` into a full slot, and the assumption `m_wait_ready` (`put |-> ready`) rules that out, so the next state is in case 2 again. ✓
3. **Safety.** In case 1 the assertion is disabled (`disable iff (rst)`); in case 2 `lost = 0`, so `!lost` holds. ✓

Without the assumption (`-D NO_ASSUME`) step 2 fails: a `put` into a full slot leaves case 2, and the verifier reports the run that does it.

16.5 A proof that lists every reachable state

For small designs the explorer enumerates every reachable state, and the proof is that set (proof kind *exhaustive*). For `01_counter.sv` it has 22 cases; a few of them:

```
in every case   c_wrap:s0 = 0 ∧ p_hold:s0 = 0 ∧ p_wrap:s0 = 0
case 1          $initstate = 1 ∧ $past(cnt) = 0 ∧ c_wrap:s1 = 0 ∧ p_hold:s1 = 0 ∧ p_wrap:s1 = 0
case 19         $initstate = 0 ∧ $past(cnt) = 8 ∧ c_wrap:s1 = 0 ∧ cnt = 9 ∧ p_hold:s1 = 0 ∧ p_wrap:s1 = 0
case 21         $initstate = 0 ∧ $past(cnt) = 9 ∧ c_wrap:s1 = 1 ∧ cnt = 0 ∧ p_hold:s1 = 0 ∧ p_wrap:s1 = 1
case 22         $initstate = 0 ∧ $past(cnt) = 9 ∧ c_wrap:s1 = 1 ∧ cnt = 9 ∧ p_hold:s1 = 1 ∧ p_wrap:s1 = 0
```

Read the checker states against the properties:

- `p_range: cnt <= 9`: no case has `cnt` above 9 (cases list `cnt = 0` … `cnt = 9`).
- `p_wrap: en && cnt == 9 |=> cnt == 0`: `p_wrap:s1 = 1` means an attempt is due *now* (the antecedent matched at the previous edge). It occurs only in case 21, where `cnt = 0`: the consequent holds.
- `p_hold: !en |=> $stable(cnt)`: `p_hold:s1 = 1` occurs only where `cnt = $past(cnt)` (cases such as 22).

The same invariant proves all three assertions, so the certificate prints it once.

16.6 A proof by an invariant the prover found

When there are too many states to list, the prover computes a compact inductive invariant. `10_lockstep.sv` has two 16-bit counters:

```
sva-verify --reset=rst --report=lockstep 10_lockstep.sv
```

Both properties are proved by one inductive invariant, which the prover found (proof kind *inductive invariant*):

```
Inv = one of these cases:
case 1   $initstate = 1
case 2   $initstate = 0 ∧ a = 0 ∧ b = 0
case 3   $initstate = 0 ∧ 0 <= b <= 12000000 ∧ b = a
```

Check it by hand:

1. **Start.** The first clock edge is the reset cycle: case 1. ✓
2. **Step.** From case 1 the reset clears both counters: case 2. From case 2 or case 3 (`a = b`), a cycle either keeps both counters or increments both (the condition `en && a < STOP` is the same for both), so `b = a` holds again; an increment happens only while `a < 10_000_000`, so `b` stays at most 10,000,000 ≤ 12,000,000. ✓
3. **Safety.** `a_same` is `b = a`, in every case but the disabled reset cycle; `a_limit` is `b <= 12_000_000`, part of case 3. ✓

No run is ever enumerated: the 16 million values of each counter are covered by three lines. The relation `b = a` is the *reason* the design is correct; add `assert property (a == b);` to your regression and any change that breaks the lock-step fails at once.

16.7 Proof kinds

Kind on the certificate	Meaning
Inductive invariant	Inv has the three properties of 16.1

Kind on the certificate	Meaning
Inductive invariant, with a backward analysis	Inv over-approximates the reachable states; a backward analysis showed that no state in Inv leads to a violation
Exact set of reachable states	Inv lists every reachable state (16.5)
`Inv = true`	the property holds from *every* state of the design; no constraint on the state is needed
no relations	decided statically, or by a bounded exploration (an initial assertion that can only fail in its first cycles)

Using an invariant in your own flow. Every relation of an inductive invariant over design signals is itself an assertion that holds: you can add it to your design (for example `assert property (a == b);`) to catch a future change that breaks the reason the design is correct, in simulation as well as in formal.

17. Safety margins and the Giolit Safety Resilience Score

A proof says *that* an assertion holds; the **safety margin** says *by how much*. With `--margins` (included in `--report`), every numeric bound of a proved assertion is tightened step by step, and the tightened assertion is proved again:

- a **value bound** such as `level <= 12` or `count <= DEPTH`: the margin is the distance between the largest value the design ever reaches and the bound, i.e. how far the invariant stays from the closest unsafe values;
- a **latency window** such as `req |-> ##[1:6] ack`: the margin is the number of cycles the window could shrink, like the slack of a timing path.

```
sva-verify --reset=rst --margins 11_margins.sv
```

```
Safety margins (how far each bound can be tightened and the assertion still holds):
  margins.a_level              level <= 4'd12         margin 3 of 13 values  ★★★☆☆
  margins.a_latency            ##[1:6]                margin 3 cycles of 6 (window)  ★★★★★
Giolit Safety Resilience Score: ★★★☆☆ (3 of 5)
```

Reading it. `a_level`: the buffer never holds more than 9 entries, 3 fewer than the limit of 12; out of the 13 values the assertion allows (0 … 12), 3 are never approached (23 %). `a_latency`: the acknowledge always comes after exactly 3 cycles, 3 cycles before the end of the window (50 %). A margin is **exact** when tightening it one step more fails with a counterexample (the verifier found the run that reaches the bound); a margin shown as `>= k` is a lower bound (the next step was not decided in time).

The score. Each bound gets stars from its relative margin (margin / the size of the allowed range):

Relative margin	Stars
50 % or more	★★★★★
25 % or more	★★★★☆
10 % or more	★★★☆☆
above 0	★★☆☆☆
0: the design reaches the bound	★☆☆☆☆

The **Giolit Safety Resilience Score** of the design is the score of its weakest bound, and appears on the certificate's cover with the table of margins.

For `10_lockstep.sv` the margin of `a_limit` is about 2,000,000 (the counters stop at 10,000,000, the assertion allows 12,000,000): 17 %, three stars. When the search runs out of time it reports what it has proved, such as `margin >= 1998848`; a margin known only to be `>= 0` is *not determined* and not rated.

What a low score means. A margin of 0 is not an error: the property holds, and the design reaches its limit, often by intent (a FIFO that fills up, a handshake that takes the full window). It tells you that any change that pushes the design toward the bound — a deeper pipeline, a slower responder, a relaxed input constraint, a parameter change — will make the assertion fail. A high score means the design has headroom: the verdict survives such changes. Properties without numeric bounds (equalities, one-hot conditions, orderings) are exact and are not rated.

18. The SDK: the verifier in your own tools

The verifier is a set of shared libraries with a C++ API (`include/giolit`, `include/vtask`); `sva-verify` is one client of it. The SDK examples in `share/giolit/sdk-examples` show how to build your own:

Example	Shows
`verify_design.cpp`	verify one file or a set of files; print every property
`counterexamples.cpp`	every failure and witness: signal values per cycle, a VCD waveform, a replay testbench
`proofs.cpp`	every proof in design terms, and as the engine stated it (JSON)
`margins.cpp`	safety margins and the resilience score
`certify.cpp`	the complete evidence bundle (certificate PDF, signed statement)
`regression.cpp`	a set of designs listed in a manifest, one CSV row per property
`low_level.cpp`	the model and the engines directly: one property, chosen engines or encoding
`license_info.cpp`	the license state, without spending a run

```
cd ~/.local/giolit-sva-verify/share/giolit/sdk-examples
./build.sh
./verify_design --reset='!rst_n' designs/fifo/fifo.sv designs/fifo/fifo_sva.sv
```

Every client follows four steps:

```
#include "giolit/License.hpp"
#include "giolit/Sva.hpp"

giolit::TokenGrant g = giolit::acquireToken();                    // 1. one run of your trial or license
giolit::SvaOptions o;
o.lower.reset = "!rst_n";                                         //    as --reset
giolit::SvaResult r = giolit::verifySva(g.token,                  // 2. every property decided, as sva-
    verify does
                                      {"fifo.sv", "fifo_sva.sv"}, o);
for (const giolit::PropertyResult& p : r.properties) {            // 3. the results
    std::cout << giolit::toString(p.status) << " " << p.name << "\n";
    if (p.hasTrace)                                               //    a counterexample or witness
        for (auto& cycle : vtask::cyclesOf(*p.task, p.cex)) ... ; //    signal -> value, per clock cycle
}
giolit::addMargins(g.token, files, o, r);                         // 4. optional: margins, proofs, the
    bundle
```

Call	Library	Gives
`acquireToken()`	`libgiolit_license`	a token for one run (all calls of the run use it)
`verifySva(token, files, options)`	`libgiolit_sva`	`SvaResult`: one `PropertyResult` per property
`addInvariants(token, files, options, result)`	`libgiolit_sva`	inductive invariants for proofs that have none to show
`addMargins(token, files, options, result)`	`libgiolit_sva`	`PropertyResult::margins`, `SvaResult::resilienceStars`
`describeProof(token, hw, property)`	`libgiolit_report`	the proof in design terms (16.3)
`waveformVcd`, `replayTestbench`	`libgiolit_report`	a run as VCD, as a testbench
`writeReport(token, hw, properties, seconds, request)`	`libgiolit_report`	the evidence bundle (section 15)
`lowerDesign`, `verifyDesign`	`libgiolit_frontend`, `libgiolit_backend`	the model and the engines, for your own schedule

A design is any number of files (modules, packages, interfaces, `bind` statements, `` `include``s); `SvaOptions::lower` takes what `sva-verify`'s options set: `top`, `reset`, `svArgs` (`-G NAME=VALUE`, `-I DIR`, `-D NAME`), `multiClock`, `encoding`. Each `acquireToken()` counts as one run of your trial or license; `regression.cpp` takes one per design.

19. Command reference

Option	Effect
`--top=MODULE`	the top-level module
`--reset=SIGNAL`	reset active in the first cycle (`rst`, `'!rst_n'`)
`--property=NAME`	verify only this property, on its cone of influence
`-p EXPR`	also assert `always EXPR` on the top module
`-G NAME=VALUE`	override a parameter of the top module
`-I DIR`, `-D NAME[=VALUE]`	include directory, macro definition
`--sv-args ...`	pass the remaining arguments to the SystemVerilog front end
`--multiclock`	exact semantics for several or derived clocks
`--encoding=MODE`	`hybrid` (default), `word`, `bit` or `race`
`--timeout=SECONDS`	total time budget (default 60)
`--json=FILE`	results and traces as JSON
`--report=DIR`	the evidence bundle: certificate or report (PDF), proofs, waveforms, replay testbenches (section 15)
`--issued-to=NAME`	with `--report`: who the certificate is for
`--margins`	the safety margin of every numeric bound and the resilience score (section 17)
`--verbose`	property source text and full traces
`--license`, `--version`, `--help`	license state, version, usage
`--activate=FILE`	apply a purchased license (asks for name, address and password)

Giolit Labs — www.giolit.com — business.giolitlabs@gmail.com